PARTICIPANT RESOURCE GUIDE

RAG

From retrieval pipelines to reasoning systems

A companion to the RAG techniques, agentic patterns. Use it to revise the ideas, share them with your team, and choose the right approach for your own projects.

What is inside

1. Why RAG, and the basics. Where LLMs fall short and how retrieval helps

2. Five RAG techniques. Naive, Hybrid, Self, Corrective and Graph RAG, with examples

3. Agentic RAG. Agents that plan, choose tools, check their work and retry

1. Why RAG, and the basics

The problem with using an LLM alone

Large language models are powerful, but they work only from what they learned during training. That creates three practical gaps for business use.

Limitation	What it means	Example
Stale knowledge	The model knows nothing after its training date	It misses a policy that changed last month
No private data	It has never seen your documents, tickets or contracts	"What is our leave policy for contractors?"
Hallucination	It gives fluent, confident answers even without a source	"Contractors get 24 paid days a year", a guess with no source

What is RAG?

Retrieval-Augmented Generation (RAG) finds the most relevant pieces of your own data at question time and gives them to the LLM together with the question. The model then answers from those pieces instead of guessing.

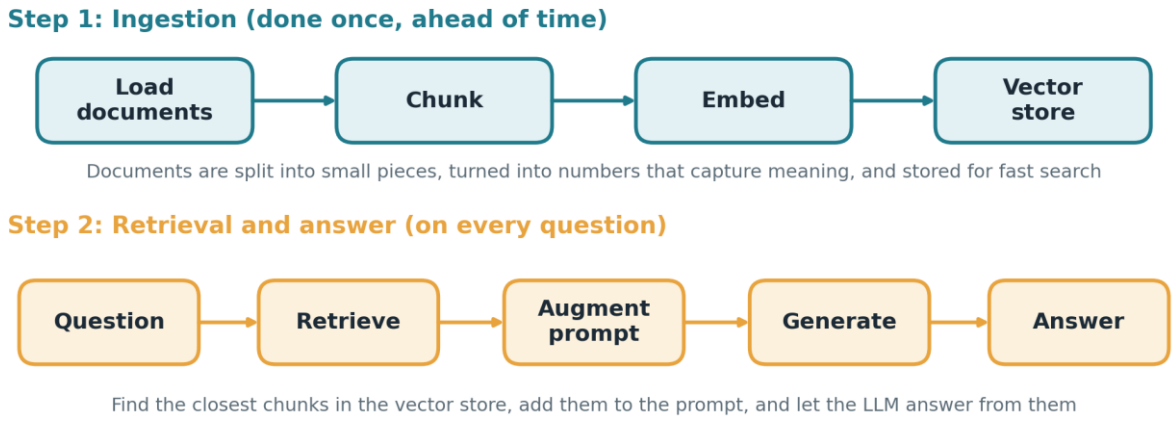


Figure 1: RAG in two steps: ingestion once, retrieval and answer on every question

- **Step 1, Ingestion:** load documents, split them into chunks, convert each chunk into an embedding, and store it in a vector store.
- **Step 2, Retrieval and answer:** embed the question, find the closest chunks, add them to the prompt, and let the LLM generate the answer.

Key terms

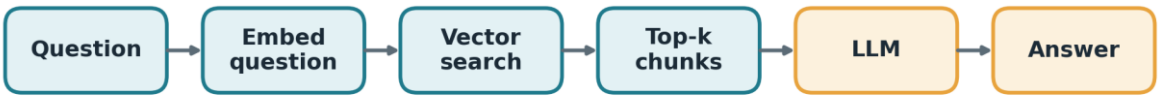
Term	Meaning
Chunk	A small piece of a document, such as a paragraph or a section
Embedding	A list of numbers that captures the meaning of a piece of text, so similar ideas sit close together
Vector store	A database that stores embeddings and finds the closest ones quickly
Retrieval (top-k)	Fetching the k chunks most similar to the question
Augmented prompt	The original question plus the retrieved chunks, sent to the LLM
Grounding	Basing the answer on retrieved sources so it can be checked

2. Five RAG techniques

Each technique fixes a weakness of the one before it. Read them in order: the problems they solve build on each other.

2.1 Naive RAG

The baseline. The question is embedded, the closest chunks are fetched once, and the LLM answers from them. It is quick to build and a good starting point.



One pass: retrieve once, generate once

Figure 2: Naive RAG: retrieve once, generate once

Good at	Struggles with
Simple FAQ lookups Fast, cheap and easy to build	Vague questions Exact codes and names Answers spread over many pages One bad chunk means a bad answer

> **Example: an HR FAQ bot**
> An employee asks, "How much notice do I give when resigning?" The bot finds the resignation section of the HR policy PDF and answers from it.

2.2 Hybrid RAG

No single way of searching finds everything. Hybrid RAG **combines several retrieval strategies** and merges their results, so the LLM sees the best of each.

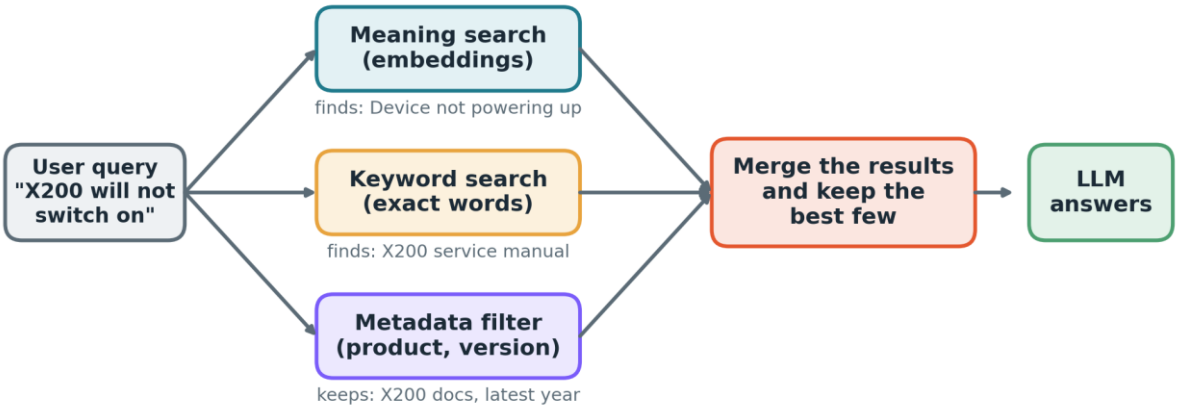


Figure 3: Hybrid RAG: several searches, one merged list

Search strategy	What it does	What it finds for the X200 question
Meaning search	Finds pages about the same idea, even with different words	"Device not powering up: troubleshooting"
Keyword search	Finds pages that contain the exact words	"X200 service manual"
Filter	Keeps only the right product and the latest version	Only X200 documents from this year

> **Example: a support assistant**
> A customer writes, "My X200 laptop will not switch on after the update." Meaning search finds general power troubleshooting. Keyword search finds the X200 manual. The filter removes other models and old versions. The merged results give the LLM the right pages.

Strengths: catches both ideas and exact names, codes and IDs. **Limits:** more than one search to set up and tune.

2.3 Self RAG

Instead of always retrieving and always trusting the result, the model **asks itself four questions along the way**. This saves effort on easy questions and catches weak answers before they reach the user.

Check	Question the model asks	Why it matters
1	Do I need to look this up?	Retrieve only when the question needs outside facts
2	Is this passage relevant?	Keep useful passages and drop the rest
3	Is my answer supported by the passage?	Check each claim against the source
4	Is the answer useful?	A final quality check before replying

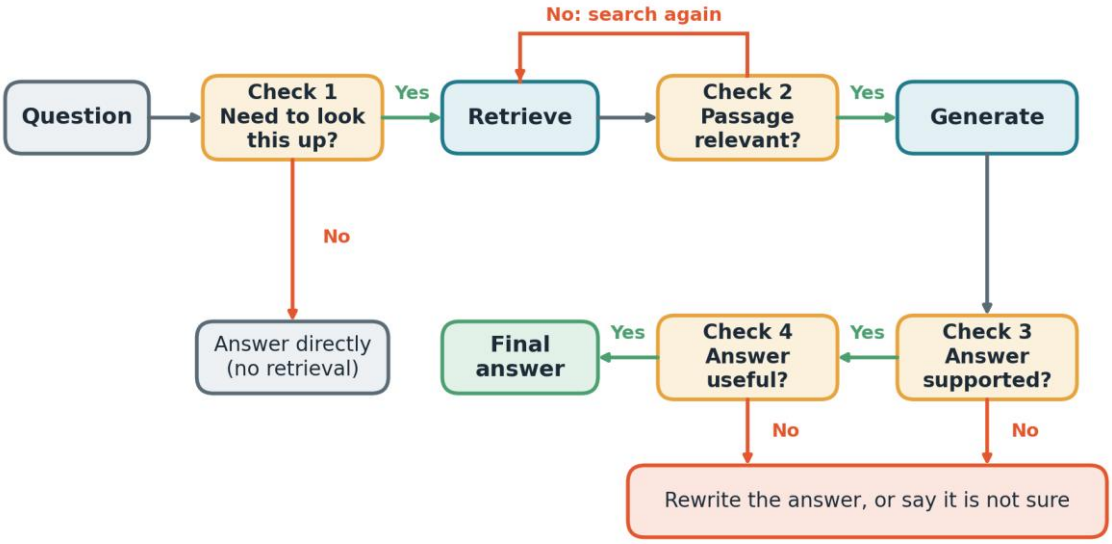


Figure 4: Self RAG: four self-checks with a path for each outcome

Self RAG in action

Situation	What the model checks	What happens
"Thanks, that helped!"	Need to look this up? No	Replies directly. No retrieval, lower cost.
"What is the refund window?"	Lookup needed, passage relevant, answer supported	Answers using the refund policy.
The retrieved page is about shipping, not refunds	Is the passage relevant? No	Drops it and searches again.
The draft says "30 days" but the policy says 14	Is the answer supported? No	Rewrites the answer, or says it is not sure.

> **Research versus practice**
> In the original research, the model is trained to produce these checks itself as special "reflection" tokens. In tools such as LangGraph, the same idea is built with grader prompts that act as the checks.

2.4 Corrective RAG

Corrective RAG adds a **grader** that scores the retrieved chunks before the LLM sees them. Weak results are not passed on. The system repairs the situation, often by searching the web.

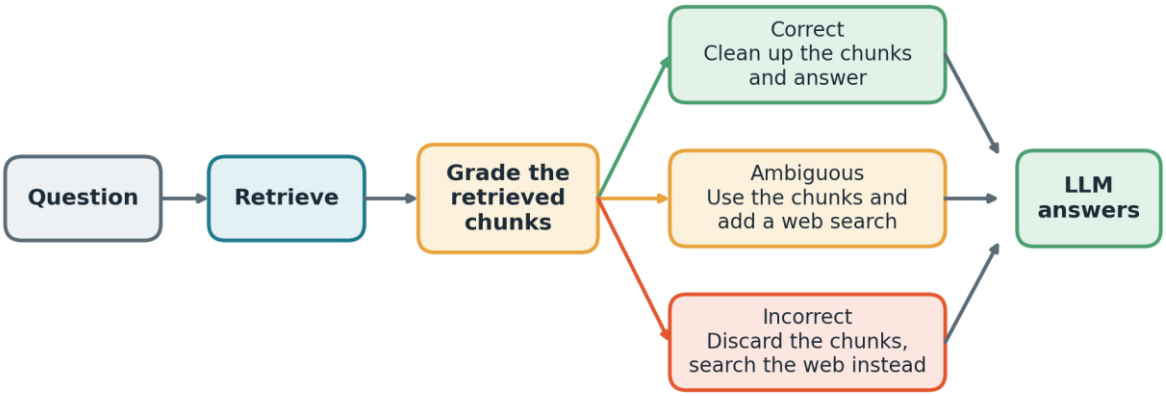


Figure 5: Corrective RAG: grade the chunks, then correct course

Grade	What the system does
Correct	Cleans up the chunks and answers
Ambiguous	Uses the chunks and adds a web search
Incorrect	Throws the chunks away and searches the web instead

> **Example: outdated internal documents**
> A user asks about a data law announced last week. The internal documents predate it, so the grader marks them incorrect and the system answers from a web search.

2.5 Graph RAG

Graph RAG stores **entities and the relationships between them** as a graph. Questions that need several connected facts become a short walk through the graph instead of a guess across separate chunks.

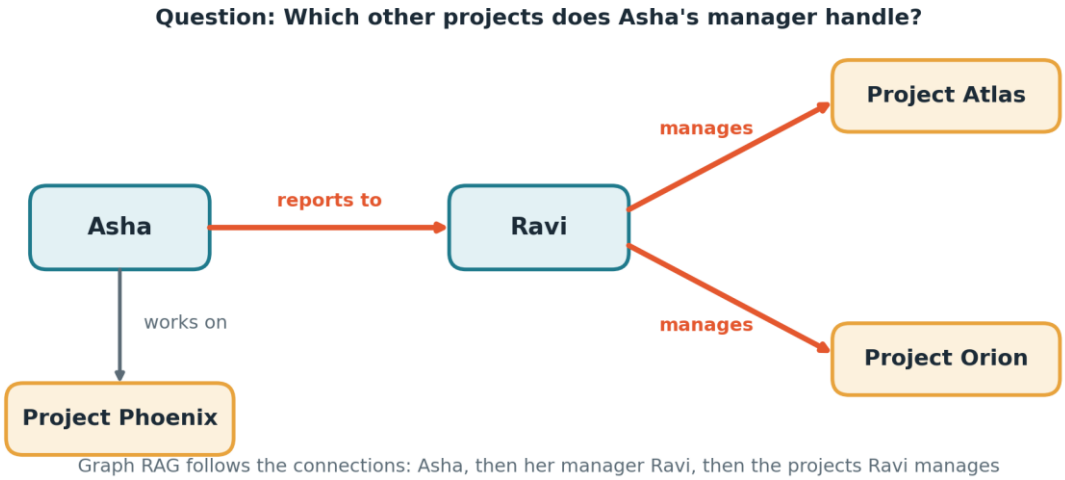


Figure 6: Graph RAG: following connections to answer a multi-step question

Approach	How it handles "Which other projects does Asha's manager handle?"
Plain RAG	Finds a page about Asha or a page about Ravi, but the link between them may sit in a different document
Graph RAG	Follows the links: Asha reports to Ravi, and Ravi manages Atlas and Orion

Strengths: multi-step questions across connected data. **Limits:** costly to build and to keep fresh as data changes.

2.6 Side by side

Technique	Fixes	Best for	Cost	Complexity
Naive	No access to private data	Simple FAQ bot	Low	Low
Hybrid	Misses exact words and codes	Product and ticket search	Low	Medium
Self	Needless or unchecked retrieval	Mixed chat and Q&A	Medium	Medium
Corrective	Wrong or outdated chunks	Knowledge that goes stale	Medium	Medium
Graph	Multi-step relationship questions	Connected data	High	High

3. Agentic RAG

3.1 From fixed pipelines to agents

All five techniques above are **fixed pipelines**: the steps are decided in advance, and the system cannot change its plan. Agentic RAG asks a different question: what if the system could decide the pipeline itself?

- An **LLM agent controls** retrieval instead of a fixed pipeline.
- It **plans**, **picks tools**, **checks** the results and **retries**.
- Retrieval becomes one tool among several.

> **Think researcher, not vending machine**
>
> **Classic RAG** is like a vending machine: press a button and get one result.
>
> **Agentic RAG** is like a researcher who breaks the question down, checks several sources, and double-checks before answering.

3.2 Classic versus agentic

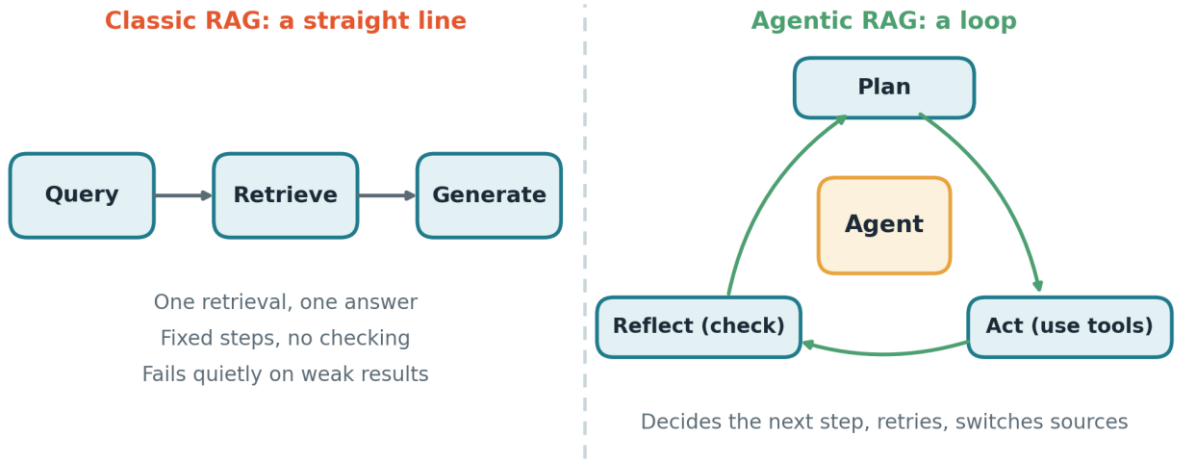


Figure 7: A straight line versus a plan, act and reflect loop

3.3 Four agentic capabilities

Capability	What it does	Example
Planning	Breaks a hard question into smaller ones	"Compare our leave and travel policy" becomes two lookups
Tool use	Picks the right source for each step	Documents, SQL, web search or an API
Reflection	Grades the result, rewrites the query, retries	Not found? Search again with new words
Memory	Remembers the conversation so far	"And for contractors?" keeps its context

3.4 Common agentic patterns

Pattern	What the agent does	Example
Router	Chooses which source should answer	HR question goes to policies, price question goes to the database
Query decomposition	Splits a question, answers each part, merges	"Compare plan A and plan B" becomes two searches
Self-correcting loop	Grades what it found and retries if weak	Rewrites the query up to a set limit
Multi-agent	Specialist agents with one coordinator	A policy agent, a finance agent and a web agent

3.5 Where the earlier techniques fit

Agentic RAG does not replace the techniques in Section 2. It decides when to use them.

Role in an agentic system	Technique	How it is used
Idea inside the loop	Self RAG	The agent checks its own answer
Idea inside the loop	Corrective RAG	The agent falls back to the web when results are weak
Tool the agent can call	Hybrid search	Better recall for names, codes and ideas
Tool the agent can call	Graph retrieval	For relationship questions
Tool the agent can call	Web, SQL and APIs	For fresh or structured data

3.6 Reference architecture

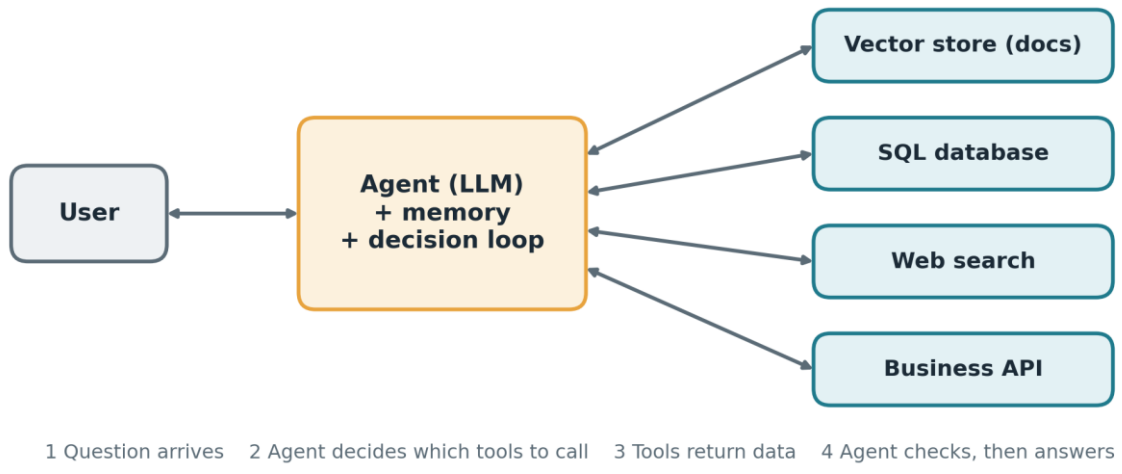


Figure 8: An agent with memory choosing among several tools

The agent sits between the user and the tools. It chooses which tool to call, reads what comes back, and decides whether to answer or try again. The numbers under the diagram show the order of events.

3.7 Trade-offs

Benefits	Costs	Guardrails
Better answers to messy questions	More latency: several LLM calls	Limit the number of iterations
Many sources behind one chat	Higher token cost	Allow only approved tools
Recovers from weak retrieval	Less predictable answers	Set timeouts and budgets
Handles multi-step questions	Harder to debug and test	Log every step for review
Easy to add new tools	Risk of endless loops	Human approval for risky actions

3.8 Which one should you pick?

Your situation	Start with
Simple FAQ over one set of documents	Naive RAG
Questions with product codes, IDs and names	Hybrid RAG
Data full of relationships: people, parts, accounts	Graph RAG
Answers must be double-checked	Self RAG or Corrective RAG
Many sources and open-ended, multi-step questions	Agentic RAG